

9

RAG Inference Pipeline

Back in *Chapter 4*, we implemented the **retrieval-augmented generation (RAG)** feature pipeline to populate the vector **database (DB)**. Within the feature pipeline, we gathered data from the data warehouse, cleaned, chunked, and embedded the documents, and, ultimately, loaded them to the vector DB. Thus, at this point, the vector DB is filled with documents and ready to be used for RAG.

Based on the RAG methodology, you can split your software architecture into three modules: one for retrieval, one to augment the prompt, and one to generate the answer. We will follow a similar pattern by implementing a retrieval module to query the vector DB. Within this module, we will implement advanced RAG techniques to optimize the search. Afterward, we won't dedicate a whole module to augmenting the prompt, as that would be overengineering, which we try to avoid. However, we will write an inference service that inputs the user query and context, builds the prompt, and calls the LLM to generate the answer. To summarize, we will implement two core Python modules, one for retrieval and one for calling the LLM using the user's input and context as input. When we glue these together, we will have an end-to-end RAG flow.

In *Chapters 5* and *6*, we fine-tuned our LLM Twin model, and in *Chapter 8*, we learned how to optimize it for inference. Thus, at this point, the LLM is ready for production. What is left is to build and deploy the two modules described above.

We will dedicate the next chapter entirely to deploying our fine-tuned LLM Twin model to AWS SageMaker, as an AWS SageMaker inference endpoint. Thus, the focus of this chapter is to dig into the advanced RAG retrieval module implementation. We have dedicated a whole chapter to the retrieval step because this is where the magic happens in an RAG system. At the retrieval step (and not when calling the LLM), you write most of the RAG inference code. This step is where you have to wrangle your data to ensure that you retrieve the most relevant data points from the vector DB. Hence, most of the advanced RAG logic goes within the retrieval step.

To sum up, in this chapter, we will cover the following topics:

- Understanding the LLM Twin's RAG inference pipeline
- Exploring the LLM Twin's advanced RAG techniques
- Implementing the LLM Twin's RAG inference pipeline

By the end of this chapter, you will know how to implement an advanced RAG retrieval module, augment a prompt using the retrieved context, and call an LLM to generate the final answer. Ultimately, you will know how to build a production-ready RAG inference pipeline end to end.

Understanding the LLM Twin's RAG inference pipeline

Before implementing the RAG inference pipeline, we want to discuss its software architecture and advanced RAG techniques. *Figure 9.1* illustrates an overview of the RAG inference flow. The inference pipeline starts with the input query, retrieves the context using the retrieval module (based on the query), and calls the LLM SageMaker service to generate the final answer.

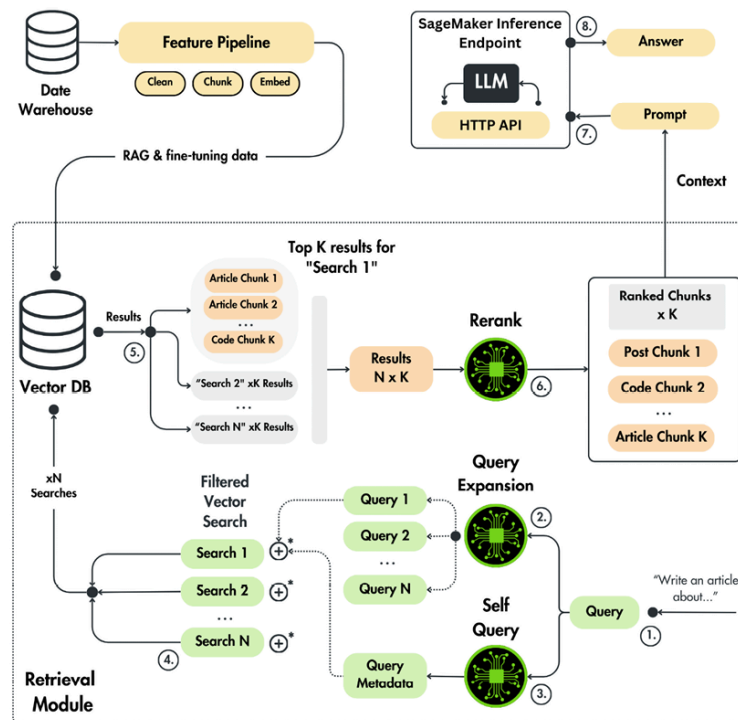


Figure 9.1: RAG inference pipeline architecture

The feature pipeline and the retrieval module, defined in *Figure 9.1*, are independent processes. The feature pipeline runs on a different machine on a schedule to populate the vector DB. At the same time, the retrieval module is called on demand, within the inference pipeline, on every user request.

By separating concerns between the two components, the vector DB is always populated with the latest data, ensuring feature freshness, while the retrieval module can access the latest features on every request. The input of the RAG retrieval module is the user's query, based on which we have to return the most relevant and similar data points from the vector DB, which will be used to guide the LLM in generating the final answer.

To fully understand the dynamics of the RAG inference pipeline, let's go through the architecture flow from *Figure 9.1* step by step:

- 1. User query:** We begin with the user who makes a query, such as "Write an article about..."
- 2. Query expansion:** We expand the initial query to generate multiple queries that reflect different aspects or interpretations of the original user query. Thus, instead of one query, we will use xN queries. By diversifying the search terms, the retrieval module increases the likeli-

hood of capturing a comprehensive set of relevant data points. This step is crucial when the original query is too narrow or vague.

3. **Self-querying:** We extract useful metadata from the original query, such as the author's name. The extracted metadata will be used as filters for the vector search operation, eliminating redundant data points from the query vector space (making the search more accurate and faster).
4. **Filtered vector search:** We embed each query and perform a similarity search to find each search's top K data points. We execute xN searches corresponding to the number of expanded queries. We call this step a filtered vector search as we leverage the metadata extracted from the self-query step as query filters.
5. **Collecting results:** We get up to xK results closest to its specific expanded query interpretation for each search operation. Further, we aggregate the results of all the xN searches, ending up with a list of $N \times K$ results containing a mix of articles, posts, and repositories chunks. The results include a broader set of potentially relevant chunks, offering multiple relevant angles based on the original query's different facets.
6. **Reranking:** To keep only the top K most relevant results from the list of $N \times K$ potential items, we must filter the list further. We will use a reranking algorithm that scores each chunk based on the relevance and importance relative to the initial user query. We will leverage a neural cross-encoder model to compute the score, a value between 0 and 1, where 1 means the result is entirely relevant to the query. Ultimately, we sort the $N \times K$ results based on the score and pick the top K items. Thus, the output is a ranked list of K chunks, with the most relevant data points situated at the top.
7. **Build the prompt and call the LLM:** We map the final list of the most relevant K chunks to a string used to build the final prompt. We create the prompt using a prompt template, the retrieved context, and the user's query. Ultimately, the augmented prompt is sent to the LLM (hosted on AWS SageMaker exposed as an API endpoint).
8. **Answer:** We are waiting for the answer to be generated. After the LLM processes the prompt, the RAG logic finishes by sending the generated response to the user.

That wraps up the overview of the RAG inference pipeline. Now, let's dig deeper into the details.

Exploring the LLM Twin's advanced RAG techniques

Now that we understand the overall flow of our RAG inference pipeline, let's explore the advanced RAG techniques we used in our retrieval module:

- **Pre-retrieval step:** Query expansion and self-querying
- **Retrieval step:** Filtered vector search
- **Post-retrieval step:** Reranking

Before digging into each method individually, let's lay down the Python interfaces we will use in this section, which are available at https://github.com/PacktPublishing/LLM-Engineers-Handbook/blob/main/llm_engineering/application/rag/base.py.

The first is a prompt template factory that standardizes how we instantiate prompt templates. As an interface, it inherits from `ABC` and exposes the `create_template()` method, which returns a `LangChain PromptTemplate` instance. Even if we avoid being heavily reliant on `LangChain`, as we want to implement everything ourselves to understand the engineering behind the scenes, some objects, such as the `PromptTemplate` class, are helpful to speed up the development without hiding too much functionality:

```
from abc import ABC, abstractmethod
from langchain.prompts import PromptTemplate
from pydantic import BaseModel
class PromptTemplateFactory(ABC, BaseModel):
    @abstractmethod
    def create_template(self) -> PromptTemplate:
        pass
```

We also want to define a `RAGStep` interface used to standardize the interface of advanced RAG steps such as query expansion and self-querying. As these steps are often dependent on other LLMs, it has a `mock` attribute to reduce costs and debugging time during development:

```
from typing import Any
from llm_engineering.domain.queries import Query
class RAGStep(ABC):
    def __init__(self, mock: bool = False) -> None:
        self._mock = mock
    @abstractmethod
    def generate(self, query: Query, *args, **kwargs) -> Any:
        pass
```

Ultimately, we must understand how we modeled the `Query` domain entity to wrap the user's input with other metadata required for advanced RAG. Thus, let's look at its implementation. First, we import the necessary classes:

```
from pydantic import UUID4, Field
from llm_engineering.domain.base import VectorBaseDocument
from llm_engineering.domain.types import DataCategory
```

Next, we define the `Query` entity class, which inherits from the `VectorBaseDocument` **object-vector mapping (OVM)** class, discussed in *Chapter 4*. Thus, each query can easily be saved or retrieved from the vector DB:

```
class Query(VectorBaseDocument):
    content: str
    author_id: UUID4 | None = None
    author_full_name: str | None = None
    metadata: dict = Field(default_factory=dict)
class Config:
    category = DataCategory.QUERIES
```

What is essential to notice are the class's attributes used to combine the user's query with a bunch of metadata fields:

- `content` : A string containing input query.
- `author_id` : An optional UUID4 identifier extracted from the query used as a filter within the vector search operation to retrieve chunks written only by a specific author
- `author_full_name` : An optional string used to query the `author_id`
- `metadata` : A dictionary for any additional metadata, initialized as an empty `dict` by default

Besides the standard definition of a domain class, we also define a `from_str()` class method to create a `Query` instance directly from a string. This allows us to standardize how we clean the query string before constructing the `query` object, such as stripping any leading or trailing whitespace and newline characters:

```
@classmethod
def from_str(cls, query: str) -> "Query":
    return Query(content=query.strip("\n "))
```

Additionally, there's an instance method called `replace_content()` used to create a new `Query` instance with updated content while retaining the original query's `id`, `author_id`, `author_full_name`, and `metadata`:

```
def replace_content(self, new_content: str) -> "Query":
    return Query(
        id=self.id,
        content=new_content,
        author_id=self.author_id,
        author_full_name=self.author_full_name,
        metadata=self.metadata,
    )
```

This can be particularly useful when modifying the query text, for example, during preprocessing or normalization, without losing the associated metadata or identifiers. Following the `Query` class, we define the `EmbeddedQuery` class:

```
class EmbeddedQuery(Query):
    embedding: list[float]
    class Config:
        category = DataCategory.QUERIES
```

The `EmbeddedQuery` class extends `Query` by adding the embedding field. The `EmbeddedQuery` entity encapsulates all the data and metadata necessary to perform vector search operations on top of Qdrant (or another vector DB).

Now that we understand all the interfaces and new domain entities used within the RAG inference pipeline, let's move on to our advanced RAG pre-retrieval optimization techniques.

Advanced RAG pre-retrieval optimizations: query expansion and self-querying

We implemented two methods to optimize the pre-retrieval optimization step: query expansion and self-querying. The two methods work closely with the filtered vector search step, which we will touch on in the next section. For now, however, we will start with understanding the code for query expansion and move to implementing self-querying.

Within these two methods, we will leverage OpenAI's API to generate variations of the original query within the query expansion step and to extract the necessary metadata within the self-querying algorithm. When we wrote this book, we used `GPT-4o-mini` in all our examples, but as OpenAI's models quickly evolve, the model might get deprecated. But that's not an issue, as you can quickly change it in your `.env` file by configuring the `OPENAI_MODEL_ID` environment variable.

Query expansion

The *problem* in a typical retrieval step is that you query your vector DB using a single vector representation of your original question. This approach covers only a small area of the embedding space, which can be limiting. If the embedding doesn't contain all the required information or nuances of your query, the retrieved context may not be relevant. This means essential documents that are semantically related but not near the query vector might be overlooked.

The *solution* is based on query expansion, which offers a way to overcome this limitation. Using an LLM to generate multiple queries based on your initial question, you create various perspectives that capture different facets of your query. These expanded queries, when embedded, target other areas of the embedding space that are still relevant to your original question. This increases the likelihood of retrieving more relevant documents from the vector DB.

Implementing query expansion can be as straightforward as crafting a detailed zero-shot prompt to guide the LLM in generating these alternative queries. Thus, after implementing query expansion, instead of having only one query to search relevant context, you will have xN queries, hence xN searches.

Increasing the number of searches can impact your latency. Thus, you must experiment with the number of queries you generate to ensure the retrieval step meets your application requirements. You can also optimize the searches by parallelizing them, drastically reducing the latency, which we will do in the `ContextRetriever` class implemented at the end of this chapter.



Query expansion is also known as multi-query, but the principles are the same. For example, this is an example of LangChain's implementation called `MultiQueryRetriever`: https://python.langchain.com/docs/how_to/Multi-QueryRetriever/

Now, let's dig into the code. We begin by importing the necessary modules and classes required for query expansion:

```
from langchain_openai import ChatOpenAI
from llm_engineering.domain.queries import Query
from llm_engineering.settings import settings
```

```
from .base import RAGStep
from .prompt_templates import QueryExpansionTemplate
```

Next, we define the `QueryExpansion` class, which generates expanded query versions. The class implementation can be found at https://github.com/PacktPublishing/LLM-Engineers-Handbook/blob/main/llm_engineering/application/rag/query_expansion.py:

```
class QueryExpansion(RAGStep):
    def generate(self, query: Query, expand_to_n: int) -> list[Query]:
        assert expand_to_n > 0, f"'expand_to_n' should be greater than 0. Got {expand_to_n}"
        if self._mock:
            return [query for _ in range(expand_to_n)]
```

In the `generate` method, we first ensure that the number of expansions requested (`expand_to_n`) is greater than zero. If the instance is in mock mode (`self._mock is True`), it simply returns a list containing copies of the original query to simulate expansion without actually calling the API. If not in mock mode, we proceed to create the prompt and initialize the language model:

```
query_expansion_template = QueryExpansionTemplate()
prompt = query_expansion_template.create_template(expand_to_n - 1)
model = ChatOpenAI(model=settings.OPENAI_MODEL_ID, api_key=settings.OPENAI_API_KEY)
```

Here, we instantiate `QueryExpansionTemplate` and create a prompt tailored to generate `expand_to_n - 1` new queries (excluding the original). We initialize the `ChatOpenAI` model with the specified settings and set the temperature to 0 for deterministic output. We then create a LangChain chain by combining the prompt with the model and invoke it with the user's question:

```
chain = prompt | model
response = chain.invoke({"question": query})
result = response.content
```

By piping the prompt into the model (`prompt | model`), we set up a chain that generates expanded queries when invoked with the original query. The response from the model is captured in the `result` object. After receiving the response, we parse and clean the expanded queries:

```
queries_content = result.strip().split(query_expansion_template.separator)
queries = [query]
queries += [
    query.replace_content(stripped_content)
    for content in queries_content
    if (stripped_content := content.strip())
]
return queries
```

We split the result using the separator defined in the template to get individual queries. Starting with a list containing the original query, we append each expanded query after stripping any extra whitespace.

Finally, we define the `QueryExpansionTemplate` class, which constructs the prompt used for query expansion. The class and other prompt templates can be accessed at

https://github.com/PacktPublishing/LLM-Engineers-Handbook/blob/main/llm_engineering/application/rag/prompt_templates.py:

```
from langchain.prompts import PromptTemplate
from .base import PromptTemplateFactory
class QueryExpansionTemplate(PromptTemplateFactory):
    prompt: str = """You are an AI language model assistant. Your task is to generate
different versions of the given user question to retrieve relevant documents from
a database. By generating multiple perspectives on the user question, your goal is
to help the user overcome some of the limitations of the distance-based similarity search.
Provide these alternative questions separated by '{separator}'.
Original question: {question}"""
    @property
    def separator(self) -> str:
        return "#next-question#"
    def create_template(self, expand_to_n: int) -> PromptTemplate:
        return PromptTemplate(
            template=self.prompt,
            input_variables=["question"],
            partial_variables={
                "separator": self.separator,
                "expand_to_n": expand_to_n,
            },
        )
```

This class defines a prompt instructing the language model to generate multiple versions of the user's question. It uses placeholders like `{expand_to_n}`, `{separator}`, and `{question}` to customize the prompt.

It takes `expand_to_n` as an input parameter to define how many queries we wish to generate while we build the `PromptTemplate` instance. The `separator` property provides a unique string to split the generated queries. The `expand_to_n` and `separator` variables are passed as `partial_variables`, making them immutable at runtime. Meanwhile, the `{question}` placeholder will be changed every time the LLM chain is called.

Now that we have finished studying the query expansion implementation, let's look at an example of how to use the `QueryExpansion` class. Let's run the following code using this `python -m llm_engineering.application.rag.query_expansion` command:

```
query = Query.from_str("Write an article about the best types of advanced RAG methods")
query_expander = QueryExpansion()
expanded_queries = query_expander.generate(query, expand_to_n=3)
for expanded_query in expanded_queries:
    logger.info(expanded_query.content)
```

We get the following variations of the original query. As you can observe, the query expansion method was successful in providing more details and different perspectives of the initial query, such as highlighting the effectiveness of advanced RAG methods or the overview of these methods (remember that the first query is the original one):


```
2024-09-18 17:51:33.529 | INFO - Write an article about the best types of advanced
2024-09-18 17:51:33.529 | INFO - What are the most effective advanced RAG methods,
2024-09-18 17:51:33.529 | INFO - Can you provide an overview of the top advanced
```

Now, let's move to the next pre-retrieval optimization method: self-querying.

Self-querying

The *problem* when embedding your query into a vector space is that you cannot guarantee that all the aspects required by your use case are present with enough signal in the embedding vector. For example, you want to be 100% sure that your retrieval depends on the tags provided in the user's input. Unfortunately, you can't control the signal left within the embedding that emphasizes the tag. By embedding the query prompt alone, you can never be sure that the tags are sufficiently represented in the embedding vector or have enough signal when computing the distance against other vectors.

This problem stands for any other metadata you want to present during the search, such as IDs, names, or categories.

The *solution* is to use self-querying to extract the tags or other critical metadata within the query and use them alongside the vector search as filters. Self-querying uses an LLM to extract various metadata fields crucial for your business use case, such as tags, IDs, number of comments, likes, shares, etc. Afterward, you have complete control over how the extracted metadata is considered during retrieval. In our LLM Twin use case, we extract the author's name and use it as a filter. Self-queries work hand-in-hand with filtered vector searches, which we will explain in the next section.

Now, let's move on to the code. We begin by importing the necessary modules and classes on which our code relies:

```
from langchain_openai import ChatOpenAI
from llm_engineering.application import utils
from llm_engineering.domain.documents import UserDocument
from llm_engineering.domain.queries import Query
from llm_engineering.settings import settings
from .base import RAGStep
from .prompt_templates import SelfQueryTemplate
```

Next, we define the `SelfQuery` class, which inherits from `RAGStep` and implements the `generate()` method. The class can be found at https://github.com/PacktPublishing/LLM-Engineers-Handbook/blob/main/llm_engineering/application/rag/self_query.py:

```
class SelfQuery(RAGStep):
    def generate(self, query: Query) -> Query:
        if self._mock:
            return query
```

In the `generate()` method, we check if the `_mock` attribute is set to `True`. If it is, we will return the original query object unmodified. This

allows us to bypass calling the model while testing and debugging. If not in mock mode, we create the prompt template and initialize the language model.

```
prompt = SelfQueryTemplate().create_template()
model = ChatOpenAI(model=settings.OPENAI_MODEL_ID, api_key=settings.OPENAI_
```

Here, we instantiate the prompt using the `SelfQueryTemplate` factory class and create a `ChatOpenAI` model instance (similar to the query expansion implementation). We then combine the prompt and the model into a chain and invoke it with the user's query.

```
chain = prompt | model
response = chain.invoke({"question": query})
user_full_name = response.content.strip("\n ")
```

We extract the content from the LLM response and strip any leading or trailing whitespace to obtain the `user_full_name` value. Next, we check if the model was able to extract any user information.

```
if user_full_name == "none":
    return query
```

If the response is `"none"`, it means no user name was found in the query, so we return the original query object. If a user name is found, we will split the `user_full_name` into the `first_name` and `last_name` variables using a utility function. Then, based on the user's details, we retrieve or create a `UserDocument` user instance:

```
first_name, last_name = utils.split_user_full_name(user_full_name)
user = UserDocument.get_or_create(first_name=first_name, last_name=last_name)
```

Finally, we update the query object with the extracted author information and return it:

```
query.author_id = user.id
query.author_full_name = user.full_name
return query
```

The updated query now contains the `author_id` and `author_full_name` values, which can be used in subsequent steps of the RAG pipeline.

Let's look at the `SelfQueryTemplate` class, which defines the prompt to extract user information:

```
from langchain.prompts import PromptTemplate
from .base import PromptTemplateFactory
class SelfQueryTemplate(PromptTemplateFactory):
    prompt: str = """You are an AI language model assistant. Your task is to extract
    The required information that needs to be extracted is the user name or user id
    Your response should consist of only the extracted user name (e.g., John Doe) or
    If the user question does not contain any user name or id, you should return the
    original question.

    For example:
```

```

QUESTION 1:
My name is Paul Iusztin and I want a post about...
RESPONSE 1:
Paul Iusztin

QUESTION 2:
I want to write a post about...
RESPONSE 2:
none

QUESTION 3:
My user id is 1345256 and I want to write a post about...
RESPONSE 3:
1345256

User question: {question}"""
def create_template(self) -> PromptTemplate:
    return PromptTemplate(template=self.prompt, input_variables=["question"])

```

In the `SelfQueryTemplate` class, we define a prompt instructing the AI model to extract the *user name* or *ID* from the input question. The prompt uses few-shot learning to guide the model on how to respond in different scenarios. When the template is invoked, the `{question}` placeholder will be replaced with the actual user question.

By implementing self-querying, we ensure that critical metadata required for our use case is explicitly extracted and used during retrieval. This approach overcomes the limitations of relying solely on the semantics of the embeddings to capture all necessary aspects of a query.

Now that we've implemented the `SelfQuery` class, let's provide an example. Run the following code using the `python -m llm_engineering.application.rag.self_query` CLI command:

```

query = Query.from_str("I am Paul Iusztin. Write an article about the best type
self_query = SelfQuery()
query = self_query.generate(query)
logger.info(f"Extracted author_id: {query.author_id}")
logger.info(f"Extracted author_full_name: {query.author_full_name}")

```

We get the following results where the author's full name and ID were extracted correctly:

```

2024-09-18 18:02:10.362 | INFO - Extracted author_id: 900fec95-d621-4315-84c6-52e57
2024-09-18 18:02:10.362 | INFO - Extracted author_full_name: Paul Iusztin

```

Now that we understand how self-querying works, let's explore how it can be used together with filtered vector search within the retrieval optimization step.

Advanced RAG retrieval optimization: filtered vector search

Vector search is pivotal in retrieving relevant information based on semantic similarity. A plain vector search, however, can introduce significant challenges that affect both the accuracy and latency of information retrieval. This is primarily because it operates solely on the numerical

proximity of vector embeddings without considering the contextual or categorical nuances that might be crucial for relevance.

One of the primary issues with plain vector search is retrieving semantically similar but contextually irrelevant documents. Since vector embeddings capture general semantic meanings, they might assign high similarity scores to content that shares language patterns or topics but doesn't align with the specific intent or constraints of the query. For instance, searching for "Java" could retrieve documents about the programming language or the Indonesian island, depending solely on semantic similarity, leading to ambiguous or misleading results.

Moreover, as the size of the dataset increases, plain vector search can suffer from scalability issues. The lack of filtering means the search algorithm has to compute similarities across the entire vector space, which can significantly increase latency.

This exhaustive search slows response times and consumes more computational resources, making it inefficient for real-time or large-scale applications.

Filtered vector search emerges as a solution by filtering after additional criteria, such as metadata tags or categories, reducing the search space before computing vector similarities. By applying these filters, the search algorithm narrows the pool of potential results to those contextually aligned with the query's intent. This targeted approach enhances accuracy by eliminating irrelevant documents that might have otherwise been considered due to their semantic similarities alone.

Additionally, filtered vector search improves latency by reducing the number of comparisons the algorithm needs to perform. Working with a smaller, more relevant subset of data decreases the computational overhead, leading to faster response times. This efficiency is crucial for applications requiring real-time interactions or handling large queries.

As the metadata used within the filtered vector search is often part of the user's input, we have to extract it before querying the vector DB. That's precisely what we did during the self-query step, where we extracted the author's name to reduce the vector space only to the author's content. Thus, as we processed the query within the self-query step, it went into the pre-retrieval optimization category, whereas when the filtered vector search optimized the query, it went into the retrieval optimization bin.

For example, when using Qdrant, to add a filter that looks for a matching `author_id` within the metadata of each document, you must implement the following code:

```
from qdrant_client.models import FieldCondition, Filter, MatchValue
records = qdrant_connection.search(
    collection_name="articles",
    query_vector=query_embedding,
    limit=3,
    with_payload=True,
    query_filter= Filter(
        must=[
            FieldCondition(
                key="author_id",
                match=MatchValue(
```

```

value=str("1234"),
),
),
],
),
)

```

In essence, while plain vector search provides a foundation for semantic retrieval, its limitations can slow performance in practical applications. Filtered vector search addresses these challenges by combining the strengths of vector embeddings with contextual filtering, resulting in more accurate and efficient information retrieval in RAG systems. The last step for optimizing our RAG pipeline is to look into reranking.

Advanced RAG post-retrieval optimization: reranking

The *problem* in RAG systems is that the retrieved context may contain irrelevant chunks that only:

- **Add noise:** The retrieved context might be irrelevant, cluttering the information and potentially confusing the language model.
- **Make the prompt bigger:** Including unnecessary chunks increases the prompt size, leading to higher costs. Moreover, language models are usually biased toward the context's first and last pieces. So, if you add a large amount of context, there's a big chance it will miss the essence.
- **Be come unaligned with your question:** Chunks are retrieved based on the similarity between the query and chunk embeddings. The issue is that the embedding model might not be tuned to your question, resulting in high similarity scores for chunks that aren't entirely relevant.

The *solution* is to use reranking to order all the $N \times K$ retrieved chunks based on their relevance relative to the initial question, where the first chunk will be the most relevant and the last the least. N represents the number of searches after query expansion, while K is the number of chunks retrieved per search. Hence, we retrieve a total of $N \times K$ chunks. In RAG systems, reranking serves as a critical post-retrieval step that refines the initial results obtained from the retrieval model.

We assess each chunk's relevance to the original query by applying the reranking algorithm, which often uses advanced models like neural cross-encoders. These models evaluate the semantic similarity between the query and each chunk more accurately than initial retrieval methods based on embeddings and the cosine similarity distance, as explained in more detail in *Chapter 4* in the *An overview of advanced RAG* section.

Ultimately, we pick the top K most relevant chunks from the sorted list of $N \times K$ items based on the reranking score. Reranking works well when combined with **query expansion**. First, let's understand how reranking works without query expansion:

1. **Search for > K chunks:** Retrieve more than K chunks to have a broader pool of potentially relevant information.
2. **Reorder using rerank:** Apply reranking to this larger set to evaluate the actual relevance of each chunk relative to the query.

3. **Take top K:** Select the top K chunks to use them as context in the final prompt.

Thus, when combined with query expansion, we gather potential valuable context from multiple points in space rather than just looking for more than K samples in a single location. Now the flow looks like this:

1. **Search for $N \times K$ chunks:** Retrieve multiple sets of chunks using the expanded queries.
2. **Reorder using rerank:** Rerank all the retrieved chunks based on their relevance.
3. **Take top K:** Select the most relevant chunks for the final prompt.

Integrating reranking into the RAG pipeline enhances the quality and relevance of the retrieved context and efficiently uses computational resources. Let's look at implementing the LLM Twin's reranking step to understand what we described above, which can be accessed on GitHub at https://github.com/PacktPublishing/LLM-Engineers-Handbook/blob/main/llm_engineering/application/rag/reranking.py.

We begin by importing the necessary modules and classes for our reranking process:

```
from llm_engineering.application.networks import CrossEncoderModelSingleton
from llm_engineering.domain.embedded_chunks import EmbeddedChunk
from llm_engineering.domain.queries import Query
from .base import RAGStep
```

Next, we define the `Reranker` class, which is responsible for reranking the retrieved documents based on their relevance to the query:

```
class Reranker(RAGStep):
    def __init__(self, mock: bool = False) -> None:
        super().__init__(mock=mock)
        self._model = CrossEncoderModelSingleton()
```

In the initializer of the `Reranker` class, we instantiate our cross-encoder model by creating an instance of `CrossEncoderModelSingleton`. This is the cross-encoder model used to score the relevance of each document chunk with respect to the query.

The core functionality of the `Reranker` class is implemented in the `generate()` method:

```
def generate(self, query: Query, chunks: list[EmbeddedChunk], keep_top_k: int)
    if self._mock:
        return chunks
    query_doc_tuples = [(query.content, chunk.content) for chunk in chunks]
    scores = self._model(query_doc_tuples)
    scored_query_doc_tuples = list(zip(scores, chunks, strict=False))
    scored_query_doc_tuples.sort(key=lambda x: x[0], reverse=True)
    reranked_documents = scored_query_doc_tuples[:keep_top_k]
    reranked_documents = [doc for _, doc in reranked_documents]
    return reranked_documents
```

The `generate()` method takes a query, a list of chunks (document segments), and the number of top documents to keep (`keep_top_k`). If we're in mock mode, it simply returns the original chunks. Otherwise, it performs the following steps:

1. Creates pairs of the query content and each chunk's content
2. Uses the cross-encoder model to score each pair, assessing how well the chunk matches the query
3. Zips the scores with the corresponding chunks to create a scored list of tuples
4. Sorts this list in descending order based on the scores
5. Selects the top `keep_top_k` chunks
6. Extracts the chunks from the tuples and returns them as the reranked documents

Before defining the `CrossEncoder` class, we import the necessary components:

```
from sentence_transformers.cross_encoder import CrossEncoder
from .base import SingletonMeta
```

We import the `CrossEncoder` class from the `sentence_transformers` library, which provides the functionality for scoring text pairs. We also import `SingletonMeta` from our base module to ensure our model class follows the singleton pattern, meaning only one instance of the model exists throughout the application. Now, we define the `CrossEncoderModelSingleton` class:

```
class CrossEncoderModelSingleton(metaclass=SingletonMeta):
    def __init__(
        self,
        model_id: str = settings.RERANKING_CROSS_ENCODER_MODEL_ID,
        device: str = settings.RAG_MODEL_DEVICE,
    ) -> None:
        """
        A singleton class that provides a pre-trained cross-encoder model for scor
        """
        self._model_id = model_id
        self._device = device
        self._model = CrossEncoder(
            model_name=self._model_id,
            device=self._device,
        )
        self._model.model.eval()
```

This class initializes the cross-encoder model using the specified `model_id` and `device` from the global `settings` loaded from the `.env` file. We set the model to evaluation mode using `self._model.model.eval()` to ensure the model is ready for inference.

The `CrossEncoderModelSingleton` class includes a callable method to score text pairs:

```
def __call__(self, pairs: list[tuple[str, str]], to_list: bool = True) -> NDArray:
    scores = self._model.predict(pairs)
    if to_list:
```

```
scores = scores.tolist()
return scores
```

The `__call__` method allows us to pass in a list of text `pairs` (each consisting of the query and a document chunk) and receive their relevance scores. The method uses the model's `predict()` function to call the model and compute the scores.

The `CrossEncoderModelSingleton` class is a wrapper over the `CrossEncoder` class, which we wrote for two purposes. The first one is for the singleton pattern, which allows us to easily access the same instance of the cross-encoder model from anywhere within the application without loading the model in memory every time we need it. The second reason is that by writing our wrapper, we defined our interface for a cross-encoder model (or any other model used for reranking). This makes the code future-proof as in case we need a different implementation or strategy for reranking, for example, using an API, we only have to write a different wrapper that follows the same interface and swap the old class with the new one. Thus, we can introduce new reranking methods without touching the rest of the code.

We now understand all the advanced RAG techniques used within our architecture. In the next section, we will examine the `ContextRetriever` class that connects all these methods and explain how to use the retrieval module with an LLM for an end-to-end RAG inference pipeline.

Implementing the LLM Twin's RAG inference pipeline

As explained at the beginning of this chapter, the RAG inference pipeline can mainly be divided into three parts: the retrieval module, the prompt creation, and the answer generation, which boils down to calling an LLM with the augmented prompt. In this section, our primary focus will be implementing the retrieval module, where most of the code and logic go. Afterward, we will look at how to build the final prompt using the retrieved context and user query.

Ultimately, we will examine how to combine the retrieval module, prompt creation logic, and the LLM to capture an end-to-end RAG workflow. Unfortunately, we won't be able to test out the LLM until we finish *Chapter 10*, as we haven't deployed our fine-tuned LLM Twin module to AWS SageMaker.

Thus, by the end of this section, you will learn how to implement the RAG inference pipeline, which you can test out end to end only after finishing *Chapter 10*. Now, let's start by looking at the implementation of the retrieval module.

Implementing the retrieval module

Let's dive into the `ContextRetriever` class implementation, which orchestrates the retrieval step in our RAG system by integrating all the advanced techniques we previously used: query expansion, self-querying, reranking, and filtered vector search. The class can be found on GitHub at <https://github.com/PacktPublishing/LLM-Engineers->

[Handbook/blob/main/llm_engineering/application/rag/retriever.py](https://learning.oreilly.com/library/view/llm-engineers-handbook/9781836200079/Text/Chapter_09.xhtml).

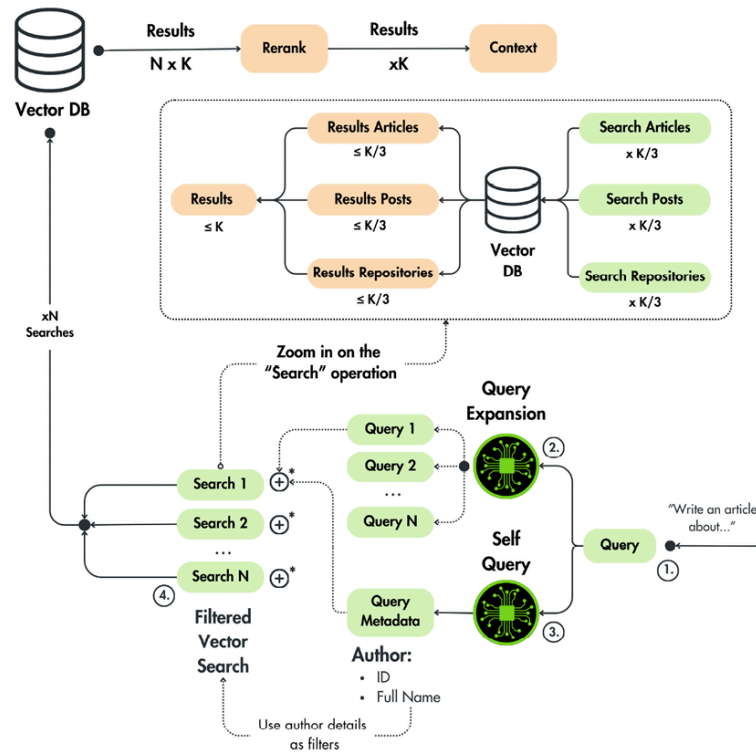


Figure 9.2: Search logic of the RAG retrieval module

The entry point function of the `ContextRetriever` class is the `search()` method, which calls all the advanced steps discussed in this chapter. Figure 9.2 shows in more detail how the search method glues together all the steps required to search results similar to the user's query. It highlights how the extracted author details from the self-query step are used within the filtered vector search. Also, it zooms in on the search operation itself, where, for each query, we do three searches to the vector DB, looking for articles, posts, or repositories similar to the query. For each search (out of N searches), we want to retrieve a maximum of K results. Thus, we retrieve a maximum of $K/3$ items for each data category (as we have three categories). Therefore, when summed up, we will have a list of $\leq K$ chunks. The retrieved list is $\leq K$ (and not equal to K) when a particular data category or more returns $< K/3$ items after applying the author filters due to missing chunks for that specific author or data category.

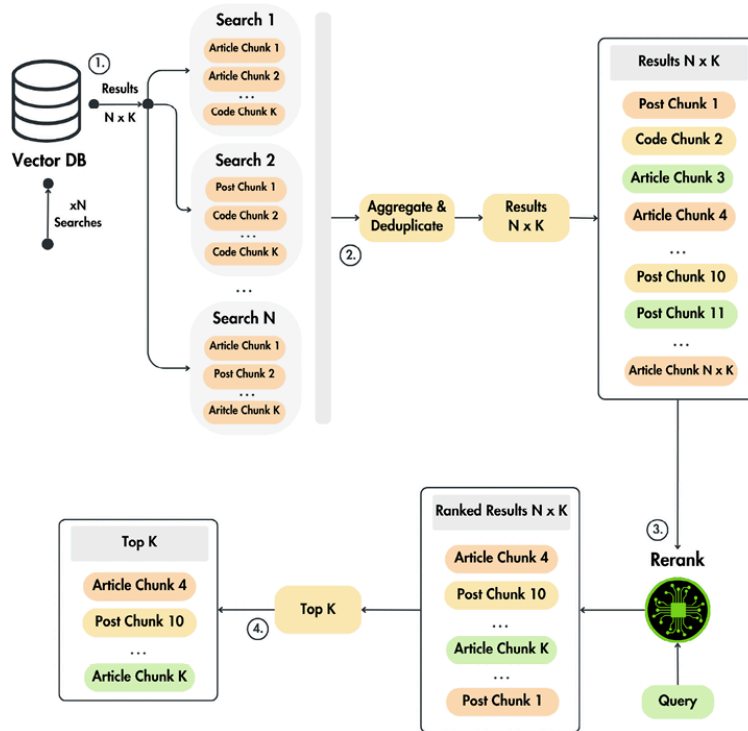


Figure 9.3: Processing the results flow of the RAG retrieval module

Figure 9.3 illustrates how we process the results returned by the xN searches. As each search returns $\leq K$ items, we will end up with $\leq N \times K$ chunks that we aggregate into a single list. As some results might overlap between searchers, we must deduplicate the aggregated list to ensure each chunk is unique. Ultimately, we send the results to the rerank model, order them based on their reranking score, and pick the **most** relevant top K chunks we will use as context for RAG.

Let's understand how everything from Figures 9.2 and 9.3 is implemented in the `ContextRetriever` class. First, we initialize the class by setting up instances of the `QueryExpansion`, `SelfQuery`, and `Reranker` classes:

```
class ContextRetriever:
    def __init__(self, mock: bool = False) -> None:
        self._query_expander = QueryExpansion(mock=mock)
        self._metadata_extractor = SelfQuery(mock=mock)
        self._reranker = Reranker(mock=mock)
```

In the `search()` method, we convert the user's input string into a query object. We then use the `SelfQuery` instance to extract the `author_id` and `author_full_name` from the query:

```
def search(
    self,
    query: str,
    k: int = 3,
    expand_to_n_queries: int = 3,
) -> list:
    query_model = Query.from_str(query)
    query_model = self._metadata_extractor.generate(query_model)
    logger.info(
        "Successfully extracted the author_id from the query.",
```

```

        author_id=query_model.author_id,
    )

```

Next, we expand the query to generate multiple semantically similar queries using the `QueryExpansion` instance:

```

n_generated_queries = self._query_expander.generate(query_model, expand_to=
logger.info(
    "Successfully generated queries for search.",
    num_queries=len(n_generated_queries),
)

```

We then perform the search concurrently for all expanded queries using a thread pool. Each query is processed by the `_search()` method, which we'll explore shortly. The results are flattened, deduplicated, and collected into a single list:

```

with concurrent.futures.ThreadPoolExecutor() as executor:
    search_tasks = [executor.submit(self._search, _query_model, k) for _query_model in n_generated_queries]
    n_k_documents = [task.result() for task in concurrent.futures.as_completed(search_tasks)]
    n_k_documents = utils.misc.flatten(n_k_documents)
    n_k_documents = list(set(n_k_documents))
    logger.info("All documents retrieved successfully.", num_documents=len(n_k_documents))

```

After retrieving the documents, we rerank them based on their relevance to the original query and keep only the top k documents:

```

if len(n_k_documents) > 0:
    k_documents = self.rerank(query, chunks=n_k_documents, keep_top_k=k)
else:
    k_documents = []
return k_documents

```

The `_search()` method performs the filtered vector search across different data categories like posts, articles, and repositories. It uses the `EmbeddingDispatcher` to convert the query into an `EmbeddedQuery`, which includes the query's embedding vector and any extracted metadata:

```

def _search(self, query: Query, k: int = 3) -> list[EmbeddedChunk]:
    assert k >= 3, "k should be >= 3"
    def _search_data_category(
        data_category_odm: type[EmbeddedChunk], embedded_query: EmbeddedQuery
    ) -> list[EmbeddedChunk]:
        if embedded_query.author_id:
            query_filter = Filter(
                must=[
                    FieldCondition(
                        key="author_id",
                        match=MatchValue(
                            value=str(embedded_query.author_id),
                        ),
                    ),
                ]
            )
        else:
            query_filter = None
        return data_category_odm.search(

```

```

        query_vector=embedded_query.embedding,
        limit=k // 3,
        query_filter=query_filter,
    )
    embedded_query: EmbeddedQuery = EmbeddingDispatcher.dispatch(query)

```

We used the same `EmbeddingDispatcher` to embed the query as in the RAG feature pipeline to embed the document chunks stored in the vector DB. Using the same class ensures we use the same embedding model at ingestion and query time, which is critical for the retrieval step.

We search each data category separately by leveraging the local `_search_data_category()` function. Within the `_search_data_category()` function, we apply the filters extracted from the `embedded_query` object. For instance, if an `author_id` is present, we use it to filter the search results only to include documents from that author. The results from all categories are then combined:

```

post_chunks = _search_data_category(EmbeddedPostChunk, embedded_query)
articles_chunks = _search_data_category(EmbeddedArticleChunk, embedded_query)
repositories_chunks = _search_data_category(EmbeddedRepositoryChunk, embedded_query)
retrieved_chunks = post_chunks + articles_chunks + repositories_chunks
return retrieved_chunks

```

Finally, the `rerank()` method takes the original query and the list of retrieved documents to reorder them based on relevance:

```

def rerank(self, query: str | Query, chunks: list[EmbeddedChunk], keep_top_k: int):
    if isinstance(query, str):
        query = Query.from_str(query)
    reranked_documents = self._reranker.generate(query=query, chunks=chunks, keep_top_k=keep_top_k)
    logger.info("Documents reranked successfully.", num_documents=len(reranked_documents))
    return reranked_documents

```

Leveraging the `ContextRetriever` class, we can retrieve context from any query with only a few lines of code. For example, let's take a look at the following code snippet, where we call the entire advanced RAG architecture with a simple call to the `search()` method:

```

from loguru import logger
from llm_engineering.application.rag.retriever import ContextRetriever

query = """
    My name is Paul Iusztin.

    Could you draft a LinkedIn post discussing RAG systems?
    I'm particularly interested in:
    - how RAG works
    - how it is integrated with vector DBs and large language models (LLMs)
    """

retriever = ContextRetriever(mock=False)
documents = retriever.search(query, k=3)
logger.info("Retrieved documents:")
for rank, document in enumerate(documents):
    logger.info(f"{rank + 1}: {document}")

```

Calling the code from above using the following CLI command: `poetry poe call-rag-retrieval-module`. This outputs the following:

```
2024-09-18 19:01:50.588 | INFO - Retrieved documents:
2024-09-18 19:01:50.588 | INFO - 1: id=UUID('541d6c22-d15a-4e6a-924a-68b7b1e0a330')
2024-09-18 19:01:50.588 | INFO - 2: id=UUID('5ce78438-1314-4874-8a5a-04f5fcf0cb21')
2024-09-18 19:02:45.729 | INFO - 3: id=UUID('0405a5da-4686-428a-91ca-446b8e0446ff')
com/decodingml/the-4-advanced-rag-algorithms-you-must-know-to-implement-5d0c7f1199c
```

As you can observe in the output above, along with the retrieved content, we have access to all kinds of metadata, such as the embedding model used for retrieval or the link from which the chunk was taken. These can quickly be added to a list of references when generating the result for the user, increasing trust in the final results.

Now that we understand how the retrieval module works, let's take a final step and examine the end-to-end RAG inference pipeline.

Bringing everything together into the RAG inference pipeline

To fully implement the RAG flow, we still have to build the prompt using the context from the retrieval model and call the LLM to generate the answer. This section will discuss these two steps and wrap everything together into a single `rag()` function. The functions from this section can be accessed on GitHub at

https://github.com/PacktPublishing/LLM-Engineers-Handbook/blob/main/llm_engineering/infrastructure/inference_pipeline_api.py.

Let's start by looking at the `call_llm_service()` function, responsible for interfacing with the LLM service. It takes in a user's query and an optional context, sets up the language model endpoint, executes the inference, and returns the generated answer. The context is optional; you can call the LLM without it, as you would when interacting with any other LLM:

```
def call_llm_service(query: str, context: str | None) -> str:
    llm = LLMInferenceSagemakerEndpoint(
        endpoint_name=settings.SAGEMAKER_ENDPOINT_INFERENCE, inference_component_name=
    )
    answer = InferenceExecutor(llm, query, context).execute()
    return answer
```

This function makes an HTTP request to our fine-tuned LLM Twin model, which is hosted as an AWS SageMaker inference endpoint. We will explore all the SageMaker details in the next chapter, where we will dig into the `LLMInferenceSagemakerEndpoint` and `InferenceExecutor` classes. For now, what is essential to know is that we use this function to call our fine-tuned LLM. Still, we must highlight how the query and context, passed to the `InferenceExecutor` class, are transformed into the final prompt. We do that using a simple prompt template that is customized using the user query and retrieved context:

```
prompt = f"""
You are a content creator. Write what the user asked you to while using the provided
User query: {query}
Context: {context}
"""
```

Moving on to the `rag()` function, this is where the RAG logic comes together. It handles retrieving relevant documents based on the query, mapping the documents to the context that will be injected into the prompt, and obtaining the final answer from the LLM:

```
def rag(query: str) -> str:
    retriever = ContextRetriever(mock=False)
    documents = retriever.search(query, k=3)
    context = EmbeddedChunk.to_context(documents)
    answer = call_llm_service(query, context)
    return answer
```

As we modularized all the RAG steps into independent classes, we reduced the high-level `rag()` function to five lines of code (encapsulating all the complexities of the system) similar to what we see in tools such as LangChain, LlamaIndex, or Haystack. Instead of their high-level implementation, we learned how to build an advanced RAG service from scratch. Also, by clearly separating the responsibility of each class, we can use them like LEGOs. Thus, you can quickly call the LLM independently without context or use the retrieval module as a query engine on top of your vector DB. In the next chapter, we will see the `rag()` function in action after we deploy our fine-tuned LLM to an AWS SageMaker inference endpoint.

Before ending this chapter, we want to discuss potential improvements you could add to the RAG inference pipeline. As we are building a chatbot, the first one is to add a conversation memory that stores all the user prompts and generated answers in memory. Thus, when interacting with the chatbot, it will be aware of the whole conversation, not only the latest prompt. When prompting the LLM, along with the new user input and context, we also pass the conversation history from the memory. As the conversation history can get long, to avoid exceeding the context window or higher costs, you have to implement a way to reduce the size of your memory. As illustrated in *Figure 9.4*, the simplest one is to keep only the latest K items from your chat history. Unfortunately, using this strategy, the LLM will never be aware of the whole conversation.

Therefore, another way to add the chat history to your prompt is to keep a summary of the conversation along with the latest K replies. There are multiple ways to compute this summary, which might defeat the purpose of this book if we get into them all, but the simplest way is to always update the summary on every user prompt and generate an answer.

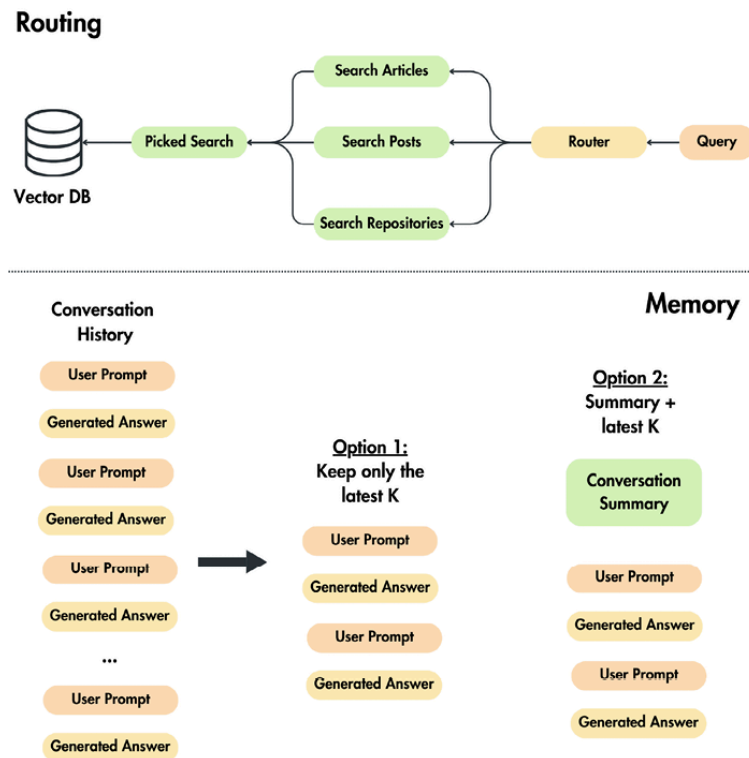


Figure 9.4: Routing and memory examples

As for each search, we send three queries to the vector DB, one for each data category. Thus, the second improvement is to add a router between the query and the search. The router will be a multi-category classifier that predicts the data categories we must retrieve for that specific query. Hence, instead of making three requests for every search, we can often reduce it to one or two. For example, if the user wants to write a theoretical paragraph about RAG for an article, then most probably, it's valuable to query only the article's collection. In this case, the router will predict the article class, which we can use to decide what collection we must query.

Another example would be if we want to illustrate a piece of code that shows how to build a RAG pipeline. In this case, the router would have to predict the article and repository data category, as we need to look up examples in both collections for an exhaustive context.

Usually, the router strategy decides what model to call based on a user's input, such as whether to use GPT-4 or a self-hosted Llama 3.1 model for that specific query. However, in our particular use case, we can adapt the router algorithm to optimize the retrieval step.

We can further optimize the retrieval using a hybrid search algorithm that combines the vector search (based on embeddings) with a keyword search algorithm, such as BM25. Search algorithms used BM25 (or similar methods) to find similar items in a DB before vector search algorithms became popular. By merging the methods, hybrid search retrieves results that match the exact terms, such as RAG, LLM, or SageMaker, and the query semantics, increasing the accuracy and relevance of your retrieved results. Fundamentally, the hybrid search algorithms follow the next mechanics:

1. **Parallel processing:** The search query is processed simultaneously through both the vector search and BM25 algorithms. Each algorithm retrieves a set of relevant documents based on its criteria.
2. **Score normalization:** The results from both searches are assigned relevance scores, which are then normalized to ensure comparability. This step is crucial because vector search and BM25 scoring mechanisms work at different scales. Thus, they can't be compared or merged without normalization.
3. **Result merging:** The normalized scores are combined, often through a weighted sum, to produce a final ranking of documents. Adjusting the weights allows for fine-tuning the emphasis on the semantic or keyword search algorithm.

To conclude, by combining the semantic and exact keyword search algorithms, you can improve the accuracy of your retrieval step. Vector search helps recognize synonyms or related concepts, ensuring that relevant information isn't overlooked due to vocabulary differences. Keyword search ensures that documents containing critical keywords are emphasized appropriately, particularly in technical fields with specific terminology.

One last improvement we can make to our RAG system is to use multi-index vector structures instead of indexing based only on the content's embedding. Let's detail how multi-indexing works. Instead of using the embeddings of a single field to do the vector search for a particular collection, it combines multiple fields.

For example, in our LLM Twin use case, we used only the content field of our articles, posts, or repositories to query the vector DB. When using a multi-index strategy, along with the content field, we could index the embeddings of the platform where the content was posted or when the content was published. This could impact the final accuracy of your retrieval as different platforms have different types of content, or more recent content is usually more relevant. Frameworks such as Superlinked make multi-indexing easy. For example, in the code snippet below, using Superlinked, we defined a multi-index on the content and platform for our article collection in just a few lines of code:

```
from superlinked.framework.common.schema.id_schema_object import IdField
from superlinked.framework.common.schema.schema import schema
from superlinked.framework.common.schema.schema_object import String
... # Other Superlinked imports.
@schema
class ArticleSchema:
    id: IdField
    platform: String
    content: String
    article = ArticleSchema()
    articles_space_content = TextSimilaritySpace(
        text=chunk(article.content, chunk_size=500, chunk_overlap=50),
        model=settings.EMBEDDING_MODEL_ID,
    )
    articles_space_plaform = CategoricalSimilaritySpace(
        category_input=article.platform,
        categories=["medium", "substack", "wordpress"],
        negative_filter=-5.0,
    )
    article_index = Index(
        [articles_space_content, articles_space_plaform],
```



```
fields=[article.author_id],  
)
```

Superlinked is a powerful Python tool for any use case that includes vector computing, such as RAG, recommender systems, and semantic search. It offers an ecosystem where you can quickly ingest data into a vector DB, write complex queries on top of it, and deploy the service as a RESTful API.

The world of LLMs and RAG is experimental, similar to any other AI domain. Thus, when building real-world products, it's important to quickly build an end-to-end solution that works but is not necessarily the best. Then, you can reiterate with various experiments until you completely optimize it for your use case. This is standard practice in the industry and lets you iterate fast while providing value to the business and gathering user feedback as quickly as possible in the product's lifecycle.

Summary

This chapter taught us how to build an advanced RAG inference pipeline. We started by looking into the software architecture of the RAG system. Then, we zoomed in on the advanced RAG methods we used within the retrieval module, such as query expansion, self-querying, filtered vector search, and reranking. Afterward, we saw how to write a modular `ContextRetriever` class that glues all the advanced RAG components under a single interface, making searching for relevant documents a breeze. Ultimately, we looked into how to connect all the missing dots, such as the retrieval, the prompt augmentation, and the LLM call, under a single RAG function that will serve as our RAG inference pipeline.

As highlighted a few times in this chapter, we couldn't test our fine-tuned LLM because we haven't deployed it yet to AWS SageMaker as an inference endpoint. Thus, in the next chapter, we will learn how to deploy the LLM to AWS SageMaker, write an inference interface to call the endpoint, and implement a FastAPI web server to serve as our business layer.

References

- *A real-time retrieval system for social media data* | VectorHub by SuperLinked. (n.d.). <https://superlinked.com/vectorhub/articles/real-time-retrieval-system-social-media-data>
- *Building a Router from Scratch - LlamaIndex*. (n.d.). https://docs.lamaindex.ai/en/stable/examples/low_level/router/
- *How to add memory to chatbots* | LangChain. (n.d.). https://python.langchain.com/docs/how_to/chatbots_memory/#summary-memory
- *How to do "self-querying" retrieval* | LangChain. (n.d.). https://python.langchain.com/docs/how_to/self_query/
- *How to route between sub-chains* | LangChain. (n.d.). https://python.langchain.com/docs/how_to/routing/#routing-by-semantic-similarity
- *How to use the MultiQueryRetriever* | LangChain. (n.d.). https://python.langchain.com/docs/how_to/MultiQueryRetriever/
- *Hybrid Search explained*. (2023, January 3). Weaviate. <https://weaviate.io/blog/hybrid-search-explained>

- Iusztin, P. (2024, August 20). 4 Advanced RAG Algorithms You Must Know | Decoding ML. *Medium*.
<https://medium.com/decodingml/the-4-advanced-rag-algorithms-you-must-know-to-implement-5d0c7f1199d2>
- Monigatti, L. (2024, February 19). Advanced Retrieval-Augmented Generation: From Theory to LlamaIndex Implementation. *Medium*.
<https://towardsdatascience.com/advanced-retrieval-augmented-generation-from-theory-to-llamaindex-implementation-4de1464a9930>
- *Multi-attribute search with vector embeddings* | VectorHub by Superlinked. (n.d.). <https://superlinked.com/vectorhub/articles/multi-attribute-semantic-search>
- *Optimizing RAG with Hybrid Search & Reranking* | VectorHub by Superlinked. (n.d.). <https://superlinked.com/vectorhub/articles/optimizing-rag-with-hybrid-search-reranking>
- Refactoring.Guru. (2024, January 1). *Singleton*. <https://refactoring.guru/design-patterns/singleton>
- Stoll, M. (2024, September 7). Visualize your RAG Data—Evaluate your Retrieval-Augmented Generation System with Ragas. *Medium*.
<https://towardsdatascience.com/visualize-your-rag-data-evaluate-your-retrieval-augmented-generation-system-with-ragas-fc2486308557>
- *Using LLM's for retrieval and reranking—LlamaIndex, data framework for LLM applications*. (n.d.). <https://www.llamaindex.ai/blog/using-llms-for-retrieval-and-reranking-23cf2d3a14b6>

Unlock this book's exclusive benefits now

This book comes with additional benefits designed to elevate your learning experience.

<https://www.packtpub.com/unlock/9781836200079>

Note: Have your purchase invoice ready before you begin.

Answers collapsed